



SSSMapGeneratorをご購入いただき ありがとうございました！

もしアセットが気に入っていただけたなら、レビューもよろしくお願いします！
願わくばいいものを…

ご意見、ご質問、プログラム改善の提案、追加機能の提案、バグ報告、その他ございましたら下記までご連絡ください

E-mail : sw2.26tks@gmail.com

X : https://x.com/sowa_jp

▼SSSMapGeneratorについて

「SSSMapGenerator」は「Slay the Spire」スタイルのマップを生成できるアセットです

非常に柔軟なカスタマイズが可能で、「Slay the Spire」のようなマップだけでなく、独自のアイデアに基づいたユニークなマップも作成できます

▼SSSMapGeneratorの注意点

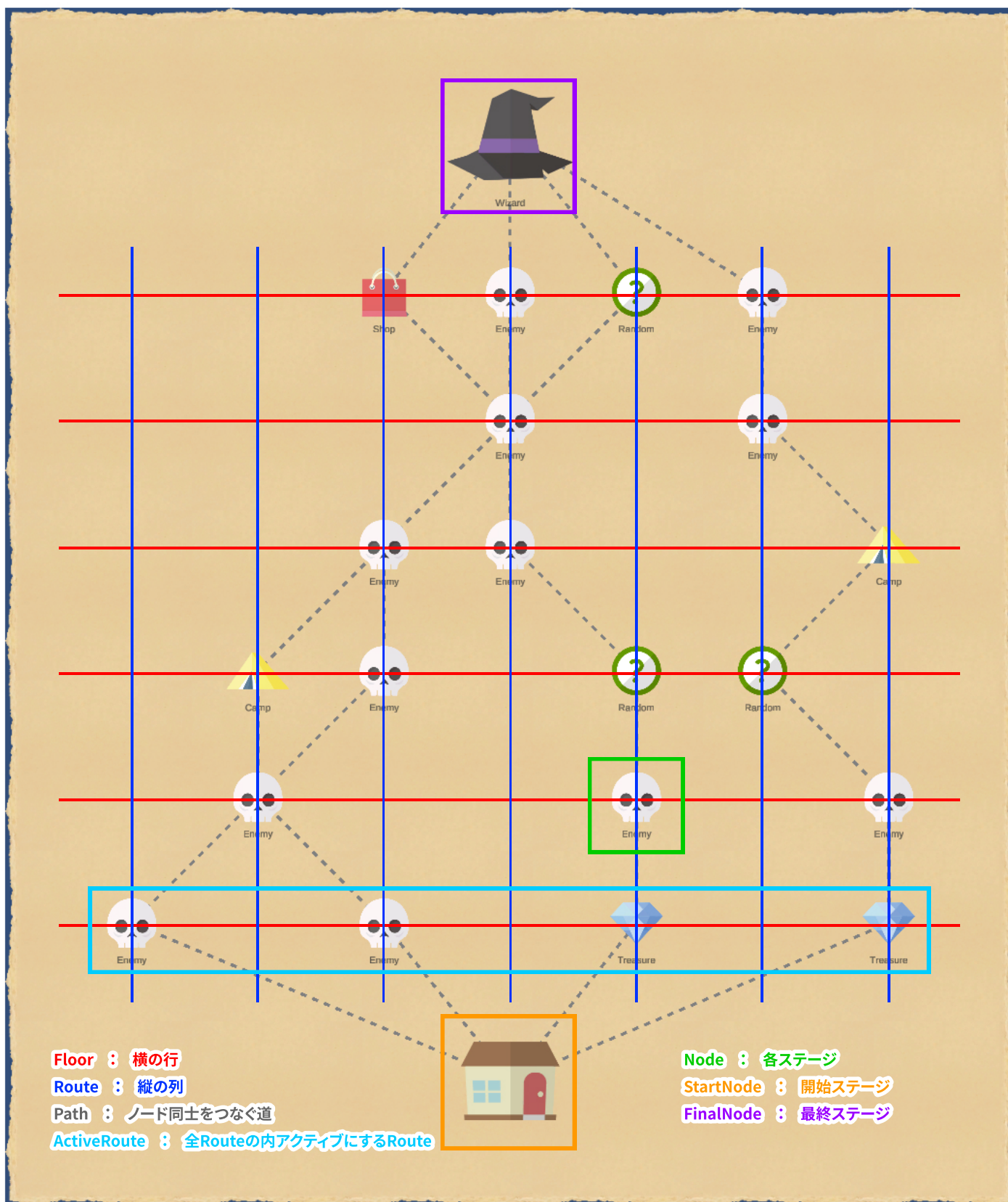
「TextMeshPro」と「InputSystem」を利用しているので事前にインストールしておいてください(必須)

セーブ/ロード及び各ノード(敵やお店)の実行には多少プログラムを触る事になりますので、

C#を全く理解できない方やビジュアルスクリプティングのみでゲームを作られている方にはおすすめしません

触る部分のプログラムに関しては非常に簡単なものになっています(各ノードの実行方法ページ参照)

▼基本

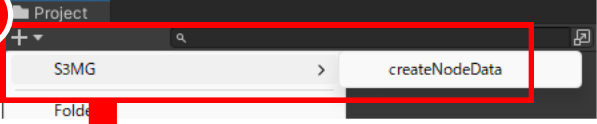


<デフォルトのマップルール>

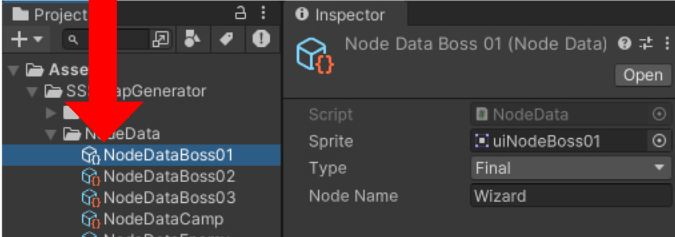
- Pathは交差しない:設定で変更可
- 同じNodeは極力連続して配置しない:設定で変更可能
 - ただし設定次第ではルールが破綻する可能性があるので最低1つは連続可能Nodeの登録必須、それでも破綻するならランダムで割り当て
- 分岐があるNodeの場合、行き先は必ず違うタイプのNodeにする:設定で変更可能
 - ただし設定次第ではルールが破綻する可能性があるので、その場合はランダムで割り当て
- マップルールが当てはまらない設定の場合は、生成不可回避の為一定の試行回数を超えると諦めてランダムでNodeを割り当て

▼クイックスタート:新規で作る場合の手順なので、Demoのオブジェクトをそのまま使っても構いません

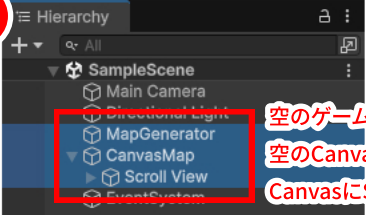
1



Projectの
S3MG > createNodeData
をクリックし、好きな数だけノードデータを作ります
これが各ステージになります
ノードデータの設定に関しては次のページで説明しますが
Node Nameは空白でも大丈夫です



2



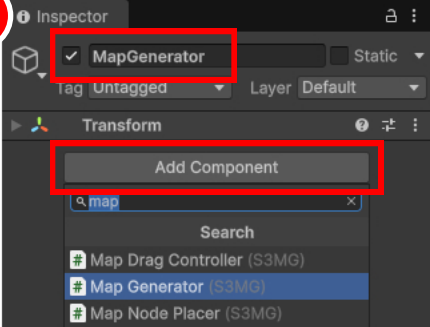
ヒエラルキーに「空のゲームオブジェクト」と「空のCanvas」と「CanvasにScrollView」を作ります
左の図ではそれぞれ
•MapGenerator
•CanvasMap
と名前をつけました

空のゲームオブジェクト
空のCanvas
CanvasにScrollView



CanvasScalerはご自身の好みの値を設定してください

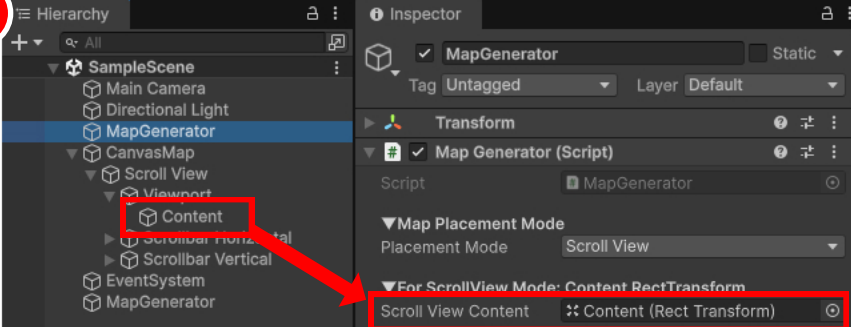
3



作った「空のゲームオブジェクト」に以下のクラスを追加します

- MapGenerator.cs
- MapNodePlacer
- MapPathRenderer
- MapDragController
- MapValidator
- NodeEventHandler
- MapSaveManager

4

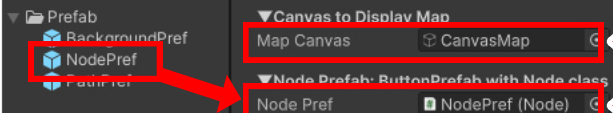


インスペクタに必須項目を設定します
必須項目は以下の4つです

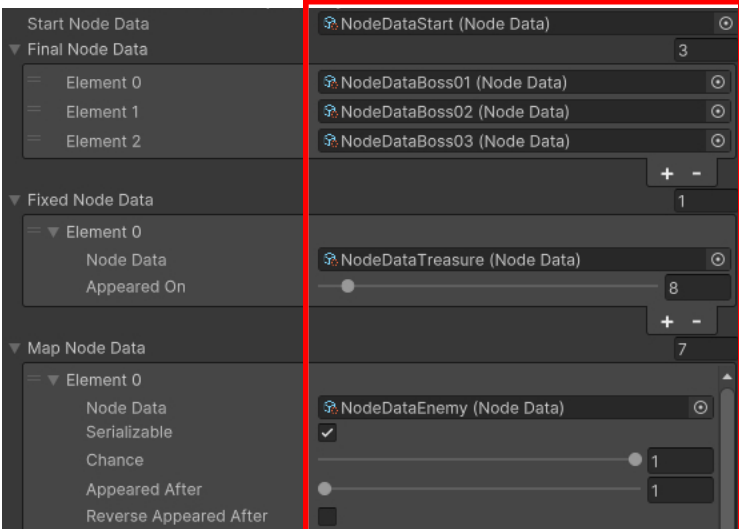
- ScrollViewのContent
- Map Canvas
- Node Pref
- 各種ノードデータ

他の設定に関しては後述します

←ScrollViewそのものではなくScrollView内のContent



←2で作った空のCanvas
←Project内のNodePref

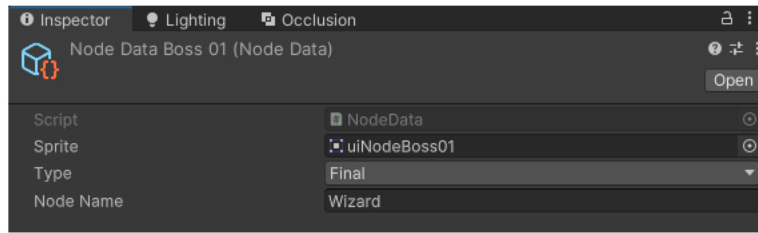


←1で作った各種ノードデータ

以上で、ゲームを開始すればマップが生成されます

※コンポーネント参照はアタッチしなくても自動取得しますが、
手動でアタッチした方が確実です

▼createNodeDataに関して



<Sprite>

ノードに表示する画像です
好きな正方形画像を登録してください



<Type>

ノードタイプです
デフォルトでは

Empty, Start, Camp, Shop, Event, Treasure, Trap, Enemy, Middle, Final, Random

の11タイプ登録してあるので好きなタイプを選択してください

上記で足りない場合は、NodeData.csクラスから独自のタイプを追加する事も可能です

もし追加した場合は、NodeEventHandler.csクラスにも処理を追加する必要があります

詳しくは「各ノードの実行方法」の「Extra」の項目で説明します

<Node Name>

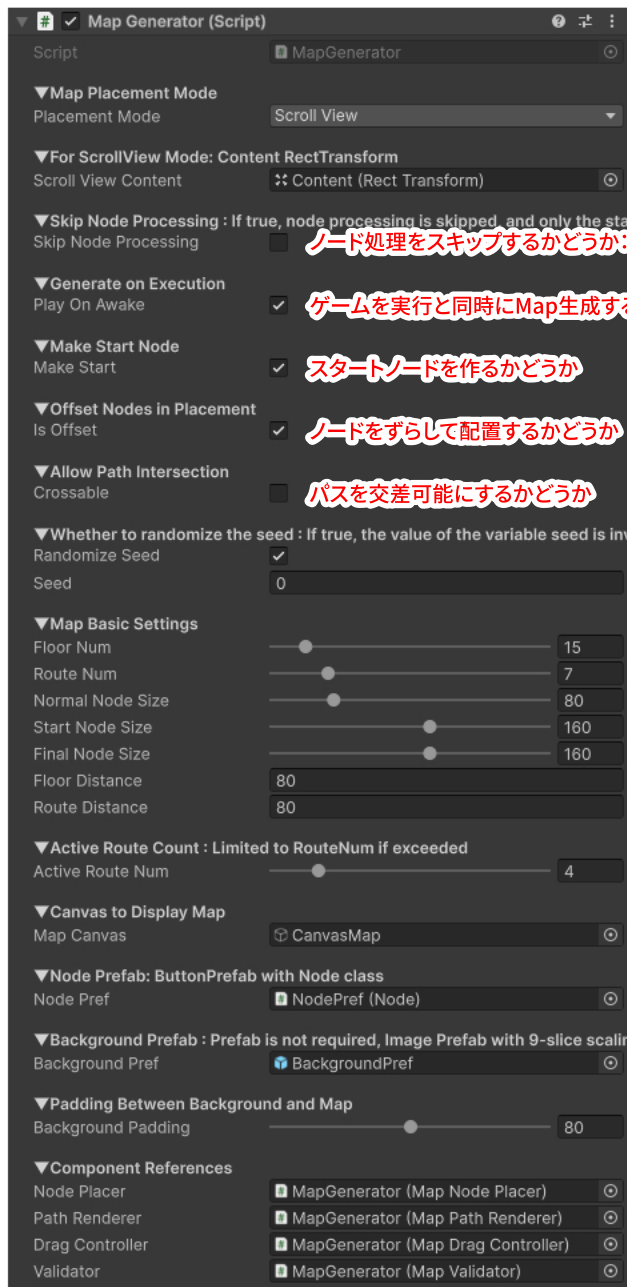
ノードの名前です

入力必須ではありませんが、入力しておくマップ上に名前を表示する事が可能になります

また、**同じTypeでも名前をつけておくと別々の処理を行う事が可能になります**

例えば「Type」を「Final」で3つ作成し、「Node Name」を「Warrior」「Wizard」「Ghost」のようにしておけばプログラムで処理を分ける事ができます

▼MapGeneratorの各種設定:画像は英語ですがスクリプト内は日本語で書いています



モード切替:StandaloneモードかScrollViewモードかを選択

ScrollViewモード時に表示するScrollView内のContent

ノード処理をスキップするかどうか:trueならノードの処理はスキップされ、ステージのみが移動

★ステージの移動だけを確認したい場合はこちらをonにしてください

ゲームを実行と同時にMap生成するかどうか

スタートノードを作るかどうか

ノードをずらして配置するかどうか

パスを交差可能にするかどうか

シード値をランダムにするかどうか:trueの場合は変数seedの値無効
シード値:「RandomizeSeed」がfalseの場合、こちらの値が使われる

フロア数

ルート数

通常ノードの大きさ

スタートノードの大きさ

ファイナルノードの大きさ

フロア間の距離

ルート間の距離

アクティブルート数:「RouteNum」より多い場合は「RouteNum」に制限

マップを表示するCanvas ★登録必須

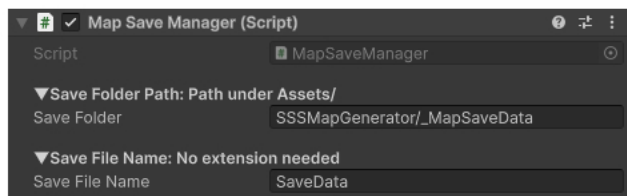
ノード用Prefab:Nodeクラスを持つButtonPrefab ★登録必須

背景Prefab:9スライスが設定されたImageのPrefab:なくても可

背景とマップの余白

各必要コンポーネント

▼MapSaveManagerの各種設定

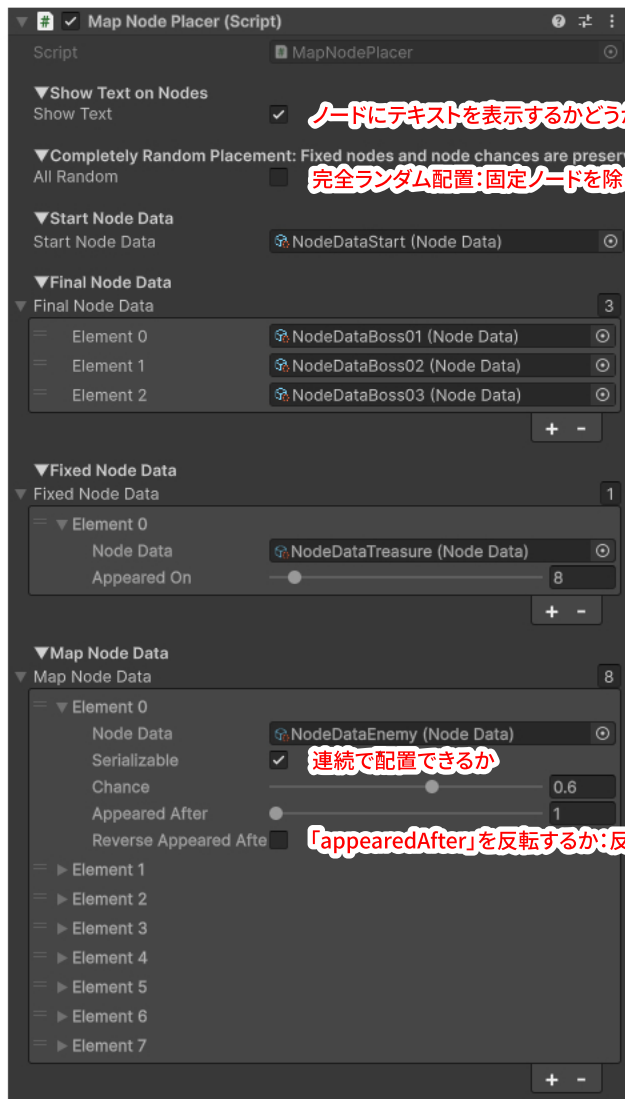


保存フォルダパス:Assets/以下のパス

保存ファイル名:拡張子不要

★セーブとロードについては「セーブとロードについて」の項目で説明します

▼MapNodePlacerの各種設定: 画像は英語ですがスクリプト内は日本語で書いています



ノードにテキストを表示するかどうか

完全ランダム配置: 固定ノードを除き、デフォルトのマップ配置ルールを適用しない

スタートノードデータ ★「MakeStart」が「true」の場合は登録必須

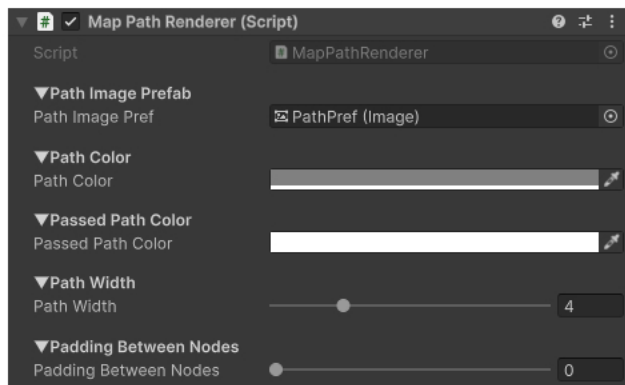
ファイナルノードデータ ★最低1つは登録必須
複数登録すればその内の1つがランダムで選択される

固定ノードデータ
複数登録可、なくても可
「Appeared on」に設定したフロアに確定で作られるが、
複数登録されていて同じフロアが設定されていれば後の方を優先

マップノードデータ ★最低1つは登録必須
複数登録推奨

出現確率
どのフロア以降から登場するか
「appearedAfter」を反転するか: 反転すると「appearedAfter」に設定したフロア以降には登場しない

▼MapPathRendererの各種設定



パスを描画するためのImageプレファブ: Shader等を設定した独自のPrefabでも可
なくても可

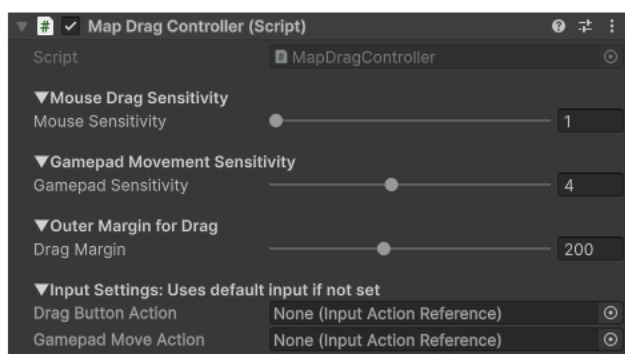
通常のパスの色

通過後のパスの色

パスの太さ

ノードとのパディング

▼MapDragControllerの各種設定



マウスのドラッグ感度

ゲームパッドの移動感度

ドラッグ時の外側の余白

マウスドラッグしたいキーのInputAction: なければマウスの左ボタンを自動で適用
ゲームパッドで動かしたいキーのInputAction:
なければゲームパッドの左スティックを自動で適用

▼各ノードの実行方法:画像は英語ですがスクリプト内は日本語で書いています

```
/*-----  
Processing for each node type: Edit here  
-----*/  
void OnStartNode(Node node){  
    Debug.Log($"Entered start node");  
}  
  
void OnCampNode(Node node){  
    Debug.Log($"Entered camp node");  
}  
  
void OnShopNode(Node node){  
    Debug.Log($"Entered shop node");  
}  
  
void OnEventNode(Node node){  
    Debug.Log($"Entered event node");  
}  
  
void OnTreasureNode(Node node){  
    Debug.Log($"Entered treasure node");  
}  
  
void OnTrapNode(Node node){  
    Debug.Log($"Entered trap node");  
}  
  
void OnEnemyNode(Node node){  
    Debug.Log($"Entered enemy node");  
}  
  
void OnMiddleNode(Node node){  
    Debug.Log($"Entered middle node");  
}  
  
void OnFinalNode(Node node){  
    Debug.Log($"Entered final node");  
    switch(node.NodeName){  
        case "Boss01":  
            Debug.Log($"Fighting Boss01");  
            break;  
        case "Boss02":  
            Debug.Log($"Fighting Boss02");  
            break;  
        default:  
            Debug.Log($"Fighting Boss03");  
            break;  
    }  
}  
  
void OnRandomNode(Node node){  
    Debug.Log($"Entered random node");  
}
```

NodeEventHandler.csクラスを開くと左図の部分があります
ここに対応するノード用の関数を用意してありますので、自由に処理を書いてください

OnFinalNode()を見ていただくとわかりますが、同じTypeでも
NodeNameによって処理を分ける事ができます

<超重要>

各ノードの処理で次のノードに移動したりマップを表示・非表示にしたいタイミングが出てくると思います

その際はMapGenerator.csクラスの以下のメソッドを実行します
シングルトンで実装されていますので、instanceを介して呼び出してください

- **ToNextNode()** 次のノードに移動するメソッド
- **ActiveMap()** マップを表示するメソッド
- **InactiveMap()** マップを非表示にするメソッド
- **Init()** 「playOnAwake」が「false」の時、
手動でマップを生成するメソッド
- **ReGenerate()** マップを再構築するメソッド

例:移動先のシーンでマップを非表示にする

using S3MG;

```
public class SampleStartSceneManager : MonoBehaviour{  
  
    void Start(){  
        MapGenerator.instance.InactiveMap();  
    }  
  
}
```

<Extra>

「createNodeDataに関して」のページで少し話しましたが、独自のタイプを追加する事も可能です

追加したい場合はまずNodeData.csクラスのenumに独自のタイプを追加します

その後、追加したenumの値を「NodeEventHandler.cs」クラスの「HandleNodeEntered()」関数内のSwitch文にも追加してください

後は他のタイプと同じように呼び出したい好きな関数を記述すればokです

▼セーブとロードについて

MapSaveManagerクラスに専用の関数を準備してあるので、そちらを呼び出してください

<例:スクリプトから呼び出す場合>

using S3MG;

//あなたが作った自作のクラス

```
public class YourClass : MonoBehaviour{
```

```
    MapSaveManager saveManager;
```

```
    void Start(){
```

```
        //MapGenerator.instanceからMapSaveManagerを取得してキャッシュ
```

```
        saveManager = MapGenerator.instance.GetComponent<MapSaveManager>();
```

```
    }
```

```
    //セーブ
```

```
    void Save(){
```

```
        saveManager?.SaveMap();
```

```
    }
```

```
    //ロード
```

```
    void Load(){
```

```
        saveManager?.LoadMap();
```

```
    }
```

```
    //セーブデータ削除
```

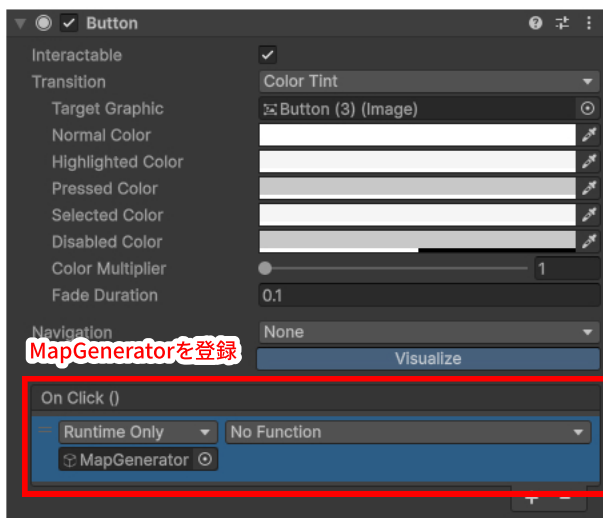
```
    void Delete(){
```

```
        saveManager?.DeleteSave();
```

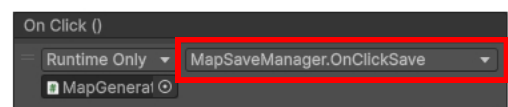
```
    }
```

```
}
```

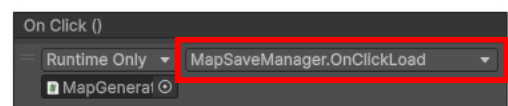
<例:ボタンから呼び出す場合>



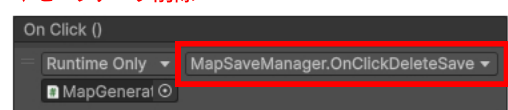
▼セーブ



▼ロード



▼セーブデータ削除



▼各Prefabについて



Prefabフォルダ内に入っているPrefabについて説明します

<BackgroundPref>

背景用のPrefabで、Imageコンポーネントだけが設定されています

「MapGenerator」の「Background Pref」に登録するPrefabですが、**使用は任意です**

登録しない場合、背景はSkyboxやカメラに設定されたカラーがそのまま表示されます

マップの広さに応じて広がるように9-sliceが設定されていますので、

もし独自の背景にしたい場合は9-sliceの画像でImageコンポーネントの「Source Image」を差し替えてください

<NodePref>

Node用のPrefabで、ButtonコンポーネントとAudioSourceコンポーネントが設定されていてNode.csクラスを持っています

「MapGenerator」の「Node Pref」に登録するPrefabで、**必須です**

Buttonコンポーネントの「Highlighted Color」や「Selected Color」は好きなカラーに変更可能です

AudioSourceコンポーネントの「AudioClip」はノードをクリックした時の音なので自由に変更可能です

Node.csクラスの「Visit Image」はノードをクリックした時にノードにアニメーションでつくマークで、自由に変更可能です

Node.csクラスの「Lerp Duration」は点滅速度、「Lerp Color」は点滅色で、自由に変更可能です

<PathPref>

Path用のPrefabで、Imageコンポーネントだけが設定されています

「MapGenerator」の「Path Image Pref」に登録するPrefabですが、**使用は任意です**

こちらを登録すると点線が使えますが、使用しない場合は単純な線になります

点線以外の独自の画像を設定する事もでき、例えば点線を動かすような独自のシェーダーを設定した画像なども設定できます

Demoでは点線をアニメーションさせるシェーダーを適用していますので、そのままお使いいただけます

▼リリースノート

詳しい内容はアセットストアのリリースログをご覧ください

▼免責

このアセットを使用して生じた不具合については、一切の責任を負いかねますのであらかじめご了承ください

▼クレジット

This assets was developed by loloop (C) All rights reserved.